창업ON 모바일 구현 설명

휴대폰에서 어떻게 보이게 만들었는지, 코드와 함께 정리한 문서

대상 저장소: SKN34-4th-3Team · 기준 커밋: 645eb09 (origin/main) · 관련 커밋: 4f5600b "모바일 앱 화면과 반응형 웹앱(/web.html) 추가"

1. 한눈에 보기

창업ON은 안드로이드/iOS 스토어에 올리는 **네이티브 앱이 아니라**, 하나의 웹사이트(React)가 휴대폰에서도 앱처럼 보이도록 만든 구조입니다. 휴대폰 대응은 세 가지 방법으로 되어 있습니다.

방법	주소	설명
① 모바일 전용 화면	`/` (index.html)	화면 폭이 768px 이하이면 PC 화면 대신 휴대폰용으로 새로 만든 화면(`src/mobile/`)을 보여줍니다.
② 반응형 웹앱	`/web.html`	PC 화면을 그대로 쓰되, CSS로 폭에 맞게 배치만 바꿉니다. 휴대폰에서는 하단 탭바가 생깁니다.
③ PWA	-	웹사이트를 휴대폰 홈 화면에 "설치"해서 앱 아이콘으로 실행할 수 있게 합니다.

> **쉽게 말하면** — 같은 가게(웹사이트)인데, 손님이 휴대폰으로 오면 휴대폰용 진열대(①)로 안내하고, 원하면 기존 진열대를 좁게 재배치(②)해서도 보여주며, 단골에게는 가게 바로가기 카드(③)를 나눠주는 셈입니다.

```
사용자가 접속
   │
   ├─ /web.html 로 접속 ──────────▶ ② 반응형 웹앱 (PC 화면 + 폭별 CSS)
   │
   └─ / 로 접속 ─▶ 화면 폭 검사(useIsMobile)
                   ├─ 768px 이하 ──▶ ① 모바일 전용 화면 (MobileApp)
                   └─ 그보다 넓음 ─▶ 기존 PC 화면

어느 쪽이든 서버 API·DB는 똑같이 사용 → 휴대폰에서 저장한 내용이 PC에서도 보임
```

2. 관련 파일 구조

파일	역할
`Frontend/src/mobile/useIsMobile.js`	지금 화면이 휴대폰 폭인지 판단
`Frontend/src/App.jsx`	휴대폰이면 MobileApp, 아니면 PC 화면을 보여주는 분기
`Frontend/src/mobile/MobileApp.jsx`	모바일 화면들의 "틀"(어느 화면을 띄울지 관리)
`Frontend/src/mobile/MHome.jsx` 외 7개	홈·로드맵·세무·지출·사업계획서·공고·마이페이지·메뉴 화면
`Frontend/src/mobile/ui.jsx`	모바일 공통 부품(바텀시트, 페이지 넘김, 토스트, 채팅 입력 등)
`Frontend/src/styles/15-mobile-app.css`	모바일 전용 화면 디자인(약 2,600줄)
`Frontend/web.html` , `src/web/`	반응형 웹앱 진입점, 하단 탭바, 웹앱 전용 CSS
`Frontend/vite.config.js`	PWA 설정, index.html·web.html 동시 빌드
`Docs/MOBILE_CSS_GUIDE.md`	모바일 CSS 작업 가이드

3. 휴대폰인지 알아채기 — useIsMobile

브라우저에는 "지금 화면이 이 조건에 맞나?"를 물어보는 `matchMedia` 기능이 있습니다. 이걸로 화면 폭이 768px 이하인지 확인하고, 창 크기가 바뀌면 다시 확인합니다.

Frontend/src/mobile/useIsMobile.js

```
// 화면 폭이 768px 이하이면 true. 창 크기를 바꾸거나 기기를 돌리면 바로 다시 판단한다.
const QUERY = '(max-width: 768px)';

export function useIsMobile() {
  const get = () => typeof window !== 'undefined' && !!window.matchMedia && window.matchMedia(QUERY).ma
tches;
  const [mobile, setMobile] = useState(get);
  useEffect(() => {
    if (!window.matchMedia) return undefined;
    const mq = window.matchMedia(QUERY);
    const onChange = () => setMobile(mq.matches);
    mq.addEventListener('change', onChange);          // 폭이 바뀌면 알려줘
    return () => mq.removeEventListener('change', onChange);
  }, []);
  return mobile;
}
```

> **쉽게 말하면** — 화면에 자를 대보고 "768px보다 좁으면 휴대폰"이라고 판단하는 함수입니다. 768px은 보통 태블릿 세로 폭 정도라, 그보다 좁으면 휴대폰으로 봅니다.

4. 화면 갈아끼우기 — App.jsx

앱의 최상위 컴포넌트 `App.jsx`에서 휴대폰이면 PC 화면 대신 `MobileApp`을 통째로 보여줍니다. 중요한 점은 **처음 열 때 한 번만** 판단한다는 것입니다. 휴대폰을 가로로 돌렸다고 화면이 PC 버전으로 바뀌면 입력하던 내용이 사라지기 때문입니다.

Frontend/src/App.jsx (발췌)

```
// 화면 폭 768px 이하에서는 모바일 전용 웹앱(src/mobile)을 보여준다. 데이터·API는 PC와 같이 쓴다.
const isMobile = useIsMobile();
// 모바일 앱 여부는 첫 로딩 때 한 번만 정한다(회전·창 크기 변경으로 화면이 바뀌어 입력이 사라지지 않게).
const [mobileApp] = useState(isMobile);

if (mobileApp && !isWeb) {
  return (
    <MobileApp
      user={user}
      roadmapDone={roadmapDone}              // 로드맵 진행률 (PC와 공유)
      setRoadmapDone={updateRoadmapDone}
      savedPolicies={savedPolicies}          // 관심 공고 (PC와 공유)
      onToggleSavedPolicy={toggleSavedPolicy}
      onLogin={() => { setAfterLogin(null); setLoginOpen(true); }}
      onLogout={() => { api.logout(); setUser(null); }}
      onProfileSaved={setUser}
    />
  );
}
// ↓ 여기서부터는 기존 PC 화면
```

> 로그인 정보, 로드맵 진행률, 관심 공고는 App이 가지고 있다가 MobileApp에 넘겨줍니다. 그래서 휴대폰과 PC가 **같은 데이터**를 보게 됩니다.

5. 모바일 앱의 틀 — MobileApp.jsx

모바일 화면은 페이지 주소를 바꾸지 않고, `screen`이라는 값 하나로 "지금 어떤 화면을 보여줄지"를 관리합니다. 화면 이동은 모두 `go()` 함수 하나로 합니다.

Frontend/src/mobile/MobileApp.jsx

```
export function MobileApp(props) {
  const { user, onLogin, initialScreen = 'home' } = props;
  const [screen, setScreen] = useState(initialScreen);   // 지금 보여줄 화면
  const [params, setParams] = useState({});              // 화면에 같이 넘길 값
  const [menuOpen, setMenuOpen] = useState(false);

  // 이동할 때 함께 넘길 값(예: 홈에서 보낸 질문, 지출관리 필터, 열어 둘 공고)을 params로 전달한다.
  const go = useCallback((key, next = {}) => {
    setMenuOpen(false);
    if (key !== 'home' && !user) {   // 홈 말고는 로그인 필요
      onLogin();
      return;
    }
    setParams({ ...next, nonce: Date.now() });
    setScreen(key);
  }, [user, onLogin]);

  const common = { ...props, params, go,
    onHome: () => go('home'), onMenu: () => setMenuOpen(true) };

  return (
    <ToastProvider>
      <div className="m-app">
        {screen === 'home'     && <MHome {...common} />}
        {screen === 'roadmap'  && <MRoadmap {...common} />}
        {screen === 'tax'      && <MTax {...common} />}
        {screen === 'expenses' && <MExpenses {...common} />}
        {screen === 'bizplan'  && <MBizplan {...common} />}
        {screen === 'gov'      && <MGov {...common} />}
        {screen === 'mypage'   && <MMyPage {...common} />}
        {menuOpen && <MMenu {...common} current={screen} onClose={() => setMenuOpen(false)} />}
      </div>
    </ToastProvider>
  );
}
```

쉽게 말하면 — TV 리모컨과 같습니다. `screen`이 채널 번호이고, `go('gov')`를 누르면 공고 채널로 바뀝니다. 채널을 바꿀 때 "이 공고를 열어줘" 같은 메모(`params`)를 함께 넘길 수 있습니다.

모바일 화면 목록

파일	화면	주요 내용
`MHome.jsx`	홈	인사 → AI 입력창(세무/공고 모드) → 기능 버튼 3×2 → 오늘의 브리핑 카드(옆으로 넘김)
`MRoadmap.jsx`	로드맵	창업 단계별 할 일 체크 + AI 코치
`MTax.jsx`	AI 세무	세무 질문 채팅
`MExpenses.jsx`	지출관리	지출 목록·필터·등록
`MBizplan.jsx`	사업계획서	사업계획서 작성·보관
`MGov.jsx`	공고	추천 공고 목록·상세·관심 저장
`MMyPage.jsx`	마이페이지	프로필, 저장한 공고·문서
`MMenu.jsx`	전체 메뉴	햄버거 버튼으로 여는 전체 메뉴

6. 휴대폰다운 사용감 만들기

모바일 화면은 **"한 화면 높이 안에 다 담고, 길게 스크롤하지 않는다"**는 원칙으로 만들었습니다. 대신 옆으로 넘기기, 페이지 넘김, 아래에서 올라오는 시트를 씁니다.

① 손가락으로 옆으로 넘기기 (홈 브리핑 카드)

Frontend/src/mobile/MHome.jsx (발췌)

```
<div className="m-brief__stack"
  onTouchStart={(e) => { touch.current = e.touches[0].clientX; }}    // 손가락 닿은 위치 기억
  onTouchEnd={(e) => {
    if (touch.current == null) return;
    const dx = e.changedTouches[0].clientX - touch.current;          // 얼마나 밀었나
    if (Math.abs(dx) > 40) move(dx < 0 ? 1 : -1);                    // 40px 넘게 밀면 다음/이전 카드
    touch.current = null;
  }}>
```

브리핑 카드에는 "마감 임박 공고(D-day)", "로드맵 다음 할 일" 등이 들어가고, 카드를 누르면 `go('gov', { openPolicyId })`처럼 해당 화면으로 바로 이동합니다.

② 화면 높이에 맞춰 목록 개수 자동 조절

휴대폰마다 화면 높이가 다르기 때문에, 목록이 들어갈 칸의 실제 높이를 재서 "한 페이지에 몇 줄이 들어가는지" 계산합니다.

Frontend/src/mobile/ui.jsx (발췌)

```
export function usePageSize(rowHeight, gap = 8, max = 99) {
  const ref = useRef(null);
  const [size, setSize] = useState(3);
  useLayoutEffect(() => {
    const el = ref.current;
    if (!el) return undefined;
    const measure = () => {
      const h = el.clientHeight;                                  // 목록 칸의 실제 높이
      if (h > 0) setSize(Math.max(1, Math.min(max, Math.floor((h + gap) / (rowHeight + gap)))));
    };
    measure();
    const ro = new ResizeObserver(measure);                       // 크기 바뀌면 다시 계산
    ro.observe(el);
    return () => ro.disconnect();
  }, [rowHeight, gap, max]);
  return [ref, size];
}

export function paginate(list, page, size) {                     // 목록을 페이지 단위로 자르기
  const pages = Math.max(1, Math.ceil(list.length / size));
  const p = Math.min(page, pages - 1);
  return { pages, page: p, items: list.slice(p * size, p * size + size) };
}
```

③ 바텀시트 (아래에서 올라오는 창)

상세 보기나 입력 폼은 휴대폰 앱처럼 화면 아래에서 올라오는 시트로 보여줍니다. 바깥을 누르거나 Esc를 누르면 닫힙니다.

Frontend/src/mobile/ui.jsx (발췌)

```
export function Sheet({ open, onClose, title, children, className = '' }) {
  useEffect(() => {
    if (!open) return undefined;
    const onKey = (e) => e.key === 'Escape' && onClose();
    window.addEventListener('keydown', onKey);
    return () => window.removeEventListener('keydown', onKey);
  }, [open, onClose]);
  if (!open) return null;
  return (
    <div className="m-sheet" onMouseDown={(e) => e.target === e.currentTarget && onClose()}>
      <div className={'m-sheet__panel ' + className} role="dialog" aria-modal="true" aria-label={title}
>
        <span className="m-sheet__handle" aria-hidden="true" />   // 위쪽 손잡이 막대
        {title && <h2 className="m-sheet__title">{title}</h2>}
        <div className="m-sheet__body">{children}</div>
      </div>
    </div>
  );
}
```

이 외에도 `ui.jsx`에 세그먼트 버튼(`Seg`), 페이지 넘김(`Pager`), 알림 토스트(`ToastProvider`), 채팅 말풍선·입력창(`ChatBubbles`, `ChatInput`), 로그인 안내(`LoginNeeded`) 같은 공통 부품이 모여 있어 각 화면이 같은 모양과 동작을 공유합니다.

7. 디자인(CSS) — PC 화면에 영향 없게

모바일 CSS의 모든 규칙은 `.m-app` 안에서만 동작하도록 작성했습니다. 그래서 모바일 디자인을 아무리 고쳐도 PC 화면은 바뀌지 않습니다.

Frontend/src/styles/15-mobile-app.css (발췌)

```
/* 모든 규칙은 .m-app 안에서만 동작하므로 PC 화면에 영향이 없다.
   원칙: 한 화면 높이(100dvh) 안에 모두 담는다. 스크롤 대신 페이지 넘김 · 세그먼트 · 접기 · 바텀시트를 쓴다.
*/

/* 모바일 앱이 떠 있을 때는 문서 자체가 스크롤되지 않게 */
html:has(.m-app), html:has(.m-app) body { overflow: hidden; }

.m-app {
  --m-primary: #2b6feb;      /* 색상을 변수로 모아 관리 */
  --m-navy: #172042;
  ...
  position: fixed;
  inset: 0;                  /* 화면 전체를 꽉 채움 */
  display: flex;
  flex-direction: column;
  overflow: hidden;
  padding-bottom: max(26px, env(safe-area-inset-bottom));  /* 아이폰 하단 홈바 영역 피하기 */
}
```

> **쉽게 말하면** — 모바일용 옷은 전부 ".m-app"이라는 옷장 안에만 넣어둔 것입니다. PC 화면은 다른 옷장을 쓰니 섞일 일이 없습니다. `safe-area-inset-bottom`은 아이폰 화면 아래쪽 막대(홈 바)에 버튼이 가려지지 않게 여백을 주는 설정입니다.

8. 반응형 웹앱 — /web.html

모바일 전용 화면과 별개로, **기존 PC 화면을 그대로 쓰면서 폭에 맞게 배치만 바꾸는** 버전도 있습니다. `web.html`로 접속하면 `src/web/main.jsx`가 실행되고, 최상위 `<html>` 태그에 `is-web`이라는 표시를 붙입니다.

Frontend/src/web/main.jsx (발췌)

```
import App from '../App.jsx';
import '../styles.css';
import './web.css';                                  // 웹앱 전용 CSS 추가

document.documentElement.classList.add('is-web');    // <html class="is-web"> 표시
...
createRoot(document.getElementById('root')).render(<App variant="web" />);
```

`web.css`의 규칙은 모두 `html.is-web` 아래에서만 동작하므로 기본 페이지(index.html)에는 영향이 없습니다. 폭에 따라 세 단계로 나눕니다.

화면 폭	배치
휴대폰 (768px 이하)	1단 배치 + **하단 탭바**
태블릿 (769~1023px)	1단/2단을 섞은 배치, 홈에 "오늘 챙길 일"
PC (1024px 이상)	기존 PC 화면 그대로

하단 탭바

Frontend/src/web/WebTabBar.jsx (발췌)

```
// 휴대폰 폭 전용 하단 탭바. 768px보다 넓으면 web.css에서 숨긴다.
// 사업계획서는 입력이 많은 화면이라 탭에서 빼고 메뉴(햄버거)로 들어간다.
const TABS = [
  { key: 'home', label: '홈' },     { key: 'roadmap', label: '로드맵' },
  { key: 'tax', label: '세무' },    { key: 'expenses', label: '지출' },
  { key: 'gov', label: '공고' },    { key: 'mypage', label: '마이' },
];

export function WebTabBar({ current, onNavigate }) {
  return (
    <nav className="wtab" aria-label="주요 기능">
      {TABS.map((t) => {
        const on = current === t.key;
        return (
          <button key={t.key} className={'wtab__item' + (on ? ' is-on' : '')}
            onClick={() => { if (!on) onNavigate(t.key); }}>
            <svg viewBox="0 0 24 24"><path d={ICON[t.key]} ... /></svg>
            <span>{t.label}</span>
          </button>
        );
      })}
    </nav>
  );
}
```

웹앱에서는 화면을 바꿀 때 주소 뒤에 `#/bizplan`처럼 기록을 남겨, 휴대폰의 **뒤로 가기** 버튼으로 이전 화면에 돌아갈 수 있게 했습니다. 저장하지 않은 사업계획서가 있으면 떠나기 전에 확인 창을 띄웁니다.

9. 앱처럼 설치하기 — PWA

PWA(Progressive Web App)는 웹사이트를 휴대폰 홈 화면에 아이콘으로 추가해서, 주소창 없이 앱처럼 실행하게 해주는 기술입니다. Vite 빌드 도구의 `vite-plugin-pwa`로 설정했습니다.

Frontend/vite.config.js (발췌)

```
VitePWA({
  registerType: 'autoUpdate',            // 새 버전 배포되면 자동으로 교체
  manifest: {
    name: '창업ON',
    short_name: '창업ON',                 // 홈 화면 아이콘 아래 이름
    start_url: '/',
    display: 'standalone',                // 주소창 없이 앱처럼 실행
    theme_color: '#f4f6fd',
    icons: [
      { src: 'pwa-192x192.png?v=2', sizes: '192x192', type: 'image/png' },
      { src: 'pwa-512x512.png?v=2', sizes: '512x512', type: 'image/png' },
      { src: 'maskable-icon-512x512.png?v=2', sizes: '512x512', type: 'image/png', purpose: 'maskable'
},
    ],
  },
  workbox: {
    globPatterns: ['**/*.{js,css,html,svg,png,ico}'],   // 화면 파일은 미리 저장(빠른 실행)
    navigateFallbackDenylist: [/^\/api\//, /^\/ppt/],   // API 응답은 저장 안 함(개인 데이터)
  },
}),

build: {
  rollupOptions: {
    input: {                              // index.html과 web.html을 함께 빌드
      main: fileURLToPath(new URL('./index.html', import.meta.url)),
      web:  fileURLToPath(new URL('./web.html',  import.meta.url)),
    },
  },
},
```

> PWA 설치는 보안 연결(HTTPS)에서만 동작합니다. 서버의 HTTPS 설정은 `Docs/AWS_DEPLOY_GUIDE.md` 12절에 정리되어 있습니다.

10. 핵심 설계 정리

설계 결정	이유
네이티브 앱 대신 웹 + PWA	코드 하나로 PC·휴대폰 모두 대응, 스토어 심사 불필요, 배포하면 바로 반영
모바일 화면을 따로 제작 (`src/mobile`)	PC 화면을 줄이기만 하면 버튼이 작고 스크롤이 길어짐 → 휴대폰에 맞게 새로 설계
모바일 여부를 첫 로딩 때 한 번만 결정	기기 회전 시 화면이 PC로 바뀌며 입력 내용이 사라지는 문제 방지
CSS를 `.m-app`, `html.is-web` 아래로 한정	모바일 수정이 PC 화면을 깨뜨리지 않게
API·데이터는 PC와 공유	휴대폰에서 저장한 공고·로드맵·지출이 PC에서도 그대로 보임
한 화면 높이 안에 담기	긴 스크롤 대신 넘기기·페이지·바텀시트로 앱 같은 사용감

11. 직접 확인하는 방법

1. `Frontend` 폴더에서 개발 서버 실행: `npm run dev` (또는 루트의 `setup.bat`)
2. 브라우저에서 `http://localhost:5173` 접속
3. **F12**(개발자 도구) → **Ctrl+Shift+M**(기기 모드)로 휴대폰 크기 선택
4. **새로고침** — 모바일 여부는 첫 로딩 때 정해지므로 크기를 바꾼 뒤 꼭 새로고침해야 합니다.
5. 반응형 웹앱은 `http://localhost:5173/web.html`로 접속해 확인